Daily Loot

Product Design Document

Turn everyday purchases into collectible game items

Version 1.0
March 2026
Status: Draft

1. Product Overview

1.1 Concept

Daily Loot is a mobile-first application that transforms everyday purchases into collectible, game-styled digital items. Users photograph items they buy, and the app uses AI to create a stylized “loot drop” image — removing backgrounds, adding cinematic bokeh, and presenting the item as if it were floating in mid-air like a video game collectible.

Each item is scored based on its cost and uniqueness relative to the user’s purchase history, producing a “happiness value” that accumulates daily. The app encourages mindful consumption and daily engagement through gamification.

1.2 Project Summary

Platform	React (web), with planned React Native expansion
Target Users	Young adults interested in gamification, journaling, and lifestyle tracking
Core Loop	Buy → Photograph → AI stylizes → Score → Accumulate daily happiness
AI Models	Vision LLM (identification) + Nano Banana (image styling) + Text LLM (scoring)
Monetization	TBD — freemium with daily item cap or subscription for unlimited

2. Image Processing Pipeline

2.1 Pipeline Overview

The image processing pipeline converts a raw photograph of a purchased item into a stylized game-item image. The pipeline is designed to minimize user friction while maintaining accuracy through a confirmation gate before the most expensive AI call.

Step	Action	Technology	Latency
1	Capture photo	Device camera / gallery upload	Instant
2	Identify item & estimate price	Vision LLM (Claude / GPT-4o / Gemini)	1–2s
3	User confirms or corrects	Frontend form (pre-filled)	User action
4	Generate stylized loot image	Nano Banana (Gemini 2.5 Flash Image)	3–8s
5	Calculate happiness score	Text LLM or rule-based logic	<1s

6	Store & display	Database + frontend	Instant

2.2 Step 1 — Item Identification

The original photo is sent to a vision-capable LLM (Claude, GPT-4o, or Gemini) with a structured prompt requesting the item name, product category, and estimated retail price. The response is returned as JSON for programmatic handling.

Example prompt: *"Identify the product in this image. Return a JSON object with fields: item_name (string), category (string, normalized e.g. 'coffee', 'electronics', 'clothing'), and estimated_price (number, USD retail estimate)."*

This step is fast (1–2 seconds) and inexpensive, making it the right place for the user confirmation gate.

2.3 Step 2 — User Confirmation

The AI's identification is presented to the user in an editable form with three fields: item name, category (dropdown), and price (numeric input with the AI estimate pre-filled). The user can accept the defaults with a single tap or make corrections. This step is critical for two reasons: it ensures the Nano Banana prompt uses an accurate item name for better image generation results, and it ensures the scoring system uses a correct price.

2.4 Step 3 — Stylized Image Generation (Nano Banana)

Nano Banana (Google's Gemini 2.5 Flash Image model) is the core image transformation engine. It accepts the original photo plus a natural language editing prompt and returns the stylized result in a single API call.

Prompt template: *"Remove all background elements, hands, tables, and supporting surfaces. Isolate the [ITEM_NAME] and make it appear floating in mid-air. Add a smooth cinematic bokeh background with soft colored light orbs. Apply [RARITY_GLOW_COLOR] rim lighting. Style it like a collectible game item with dramatic lighting and a subtle glow around the object."*

The [ITEM_NAME] and [RARITY_GLOW_COLOR] variables are injected from the confirmed item data. If the user is unsatisfied with the result, they can tap "Retry image" which re-sends the request. The user's corrected item name (if any) is already embedded in the prompt for improved accuracy on retry.

2.5 API Access & Cost

Nano Banana is accessible via fal.ai's hosted API at approximately $0.15 per edit (Pro tier), or through the Gemini API directly. For an MVP, fal.ai provides a simpler integration path. At 3–5 items per user per day, the image generation cost is $0.45–$0.75 per user per day, which should be factored into the pricing model.

2.6 Known Risks

- Object occlusion: If a hand grips a small item tightly, Nano Banana may hallucinate missing parts of the object. Fallback: prompt user to retake the photo with item on a flat surface.
- Prompt consistency: The “game item” aesthetic will vary across different input photos. The prompt template needs extensive testing across 20–30 diverse product types before launch.
- Cost at scale: Image generation is the most expensive per-request operation. Consider caching, daily item caps for free-tier users, or lower-cost model tiers for Common items.

3. Scoring System

3.1 Per-Item Happiness Value

Each item receives a happiness value from 0 to 100, composed of two equally weighted components: a cost score (0–50) and a uniqueness score (0–50).

3.2 Cost Component (50%)

The cost score is determined by a tiered bracket system based on the item's confirmed price. The tiered approach caps the influence of expensive items, preventing a single luxury purchase from dominating the daily score.

Tier	Price Range	Cost Score	Glow Color
Common	$0 – $5	10 / 50	Teal (#1D9E75)
Uncommon	$5 – $20	20 / 50	Silver (#888780)
Rare	$20 – $50	30 / 50	Gold (#EF9F27)
Epic	$50 – $200	40 / 50	Purple (#534AB7)
Legendary	$200+	50 / 50	Red/Orange (#E24B4A)

The rarity tier also determines the visual treatment of the loot card (glow color, border accent) and the Nano Banana prompt (rim lighting color).

3.3 Uniqueness Component (50%)

Uniqueness is calculated relative to the user's own purchase history using an inverse frequency decay formula:

```
uniqueness_score = 50 / (1 + previous_occurrences)
```

The category matching uses the normalized category returned by the identification LLM. For example, "iced latte" and "cappuccino" both fall under the "coffee" category. A first-time item scores the full 50 points; the second occurrence drops to 25; the third to approximately 17, and so on. This rewards variety and makes novel purchases more exciting.

3.4 Daily Happiness Value

The daily happiness value is the sum of all individual item scores logged that day, with no cap. This means logging more items always increases the daily score, which rewards engagement and the habit of noticing small purchases.

Example day: A user logs wireless earbuds (89 pts), an iced matcha latte (68 pts), a banana bunch (50 pts), and a sandwich (40 pts). Daily happiness = 247 points.

3.5 Price Determination (Hybrid)

The item price is determined through a hybrid approach: the vision LLM provides an initial estimate during identification, and the user can adjust it on the confirmation screen. The AI estimate is pre-filled to minimize friction — users only need to interact if the estimate is wrong. This balances accuracy with ease of use.

4. User Experience

4.1 Core User Flows

4.1.1 Daily Item Capture (Primary Flow)

1. User opens app and taps the floating action button (+) on the home screen.
2. Camera viewfinder opens. User takes a photo or selects from gallery.
3. AI identifies the item, estimates price, and categorizes it. Results shown on the confirmation screen.
4. User confirms or corrects the item name, category, and price.
5. User taps “Generate loot card.” Nano Banana processes the image (3–8 seconds with a loading state).
6. Loot drop screen reveals the stylized image, rarity badge, and score breakdown.
7. User can “Retry image” (re-generate) or “Save” to add the item to today’s log.

4.1.2 Browsing History (Calendar View)

The home screen displays the current month as a calendar on the upper half. Days with logged items show highlighted dots or score badges. Tapping a day loads that day’s items in the lower half as a scrollable gallery of loot cards, each showing the stylized image thumbnail, item name, rarity badge, price, and happiness points. Tapping an individual card opens a full detail view with both the original and stylized images, plus the complete score breakdown.

4.1.3 Trends View

A tab toggle on the bottom half of the home screen switches between “Items” (the loot card gallery) and “Trends” (graphical data). The trends view includes a line chart of daily happiness scores over time (filterable by week, month, or all-time), a category breakdown showing purchase patterns, and a rarity distribution chart.

4.2 Screen Structure

4.2.1 Home Screen

- Top half: Monthly calendar with item-logged indicators on active days. Current/selected day highlighted.
- Bottom half: Tab toggle (“Items” / “Trends”). Items tab shows daily total and scrollable loot cards. Trends tab shows graphs.
- Floating action button (FAB): Purple circle with + icon, centered at bottom. Primary entry point for new item capture.

4.2.2 Capture Flow (3 Screens)

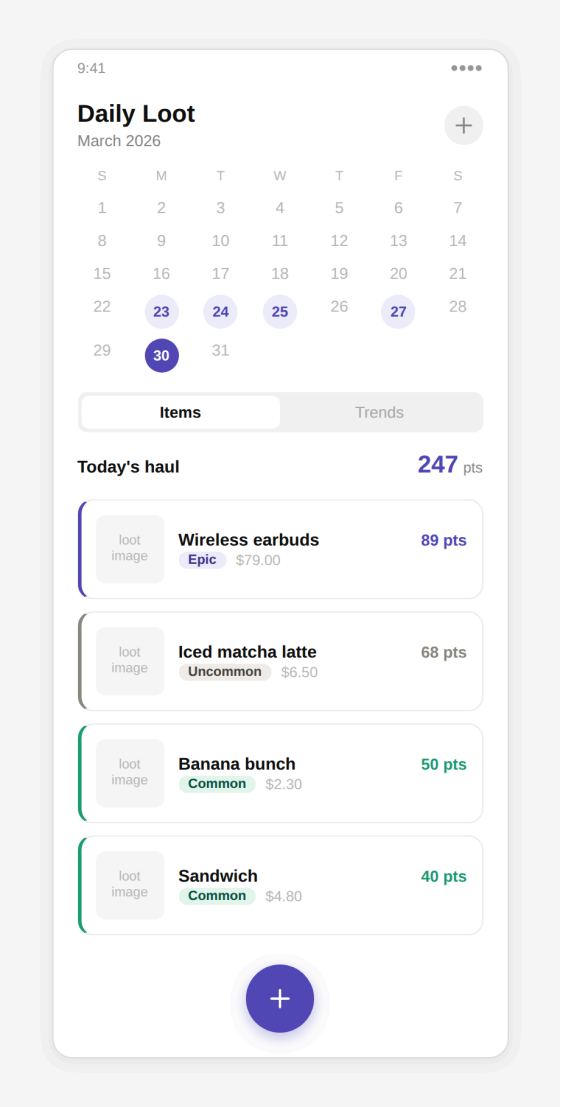
- Screen 1 — Capture: Camera viewfinder with “Take photo” and “Gallery” buttons. Cancel returns to home.

- Screen 2 — Confirm details: Photo thumbnail, editable fields for item name, category (dropdown), and price (numeric). “Generate loot card” primary action button.
- Screen 3 — Loot drop: Stylized image display with rarity badge overlay, score breakdown (cost score, uniqueness score, total), and “Retry image” / “Save” action buttons.

4.3 Wireframes

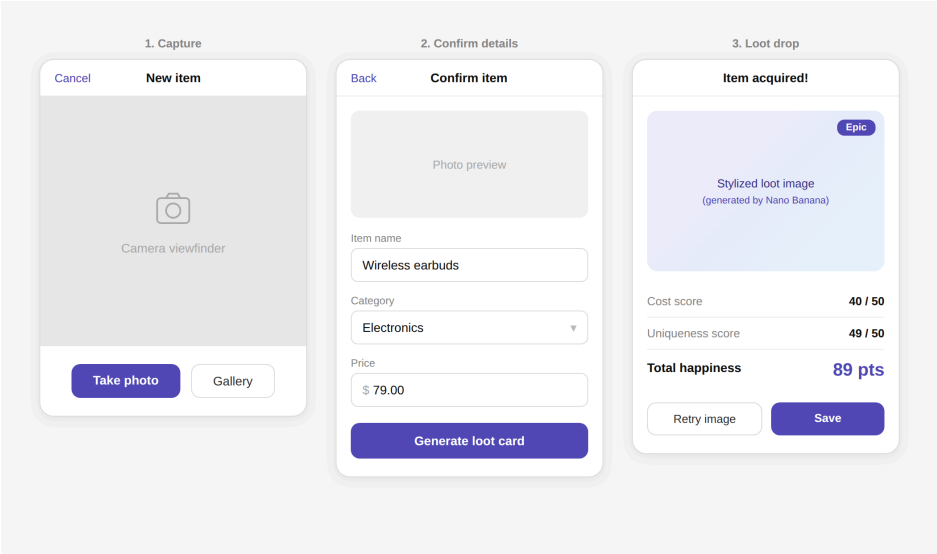
4.3.1 Home Screen

The home screen wireframe shows the calendar on the upper half with highlighted days indicating logged items, the Items/Trends tab toggle, the daily happiness total, and the scrollable loot card gallery with rarity-colored borders. The floating action button is centered at the bottom.



4.3.2 Capture Flow

The capture flow consists of three screens presented sequentially. Screen 1 shows the camera viewfinder with capture and gallery options. Screen 2 presents the AI-identified item details in an editable form for user confirmation. Screen 3 reveals the final loot drop with the stylized image, rarity badge, score breakdown, and save/retry actions.



4.4 Visual Design Language

The app uses a clean, minimal aesthetic with rarity-driven color accents. Each rarity tier has an associated color that appears consistently across loot card borders, glow effects in the stylized image, score displays, and calendar indicators. The primary brand color is purple (#534AB7), matching the “Epic” rarity tier, used for the FAB, selected states, and key interactive elements.

5. Data Model

5.1 Item Record

Each logged item is stored as a single record with the following fields:

id	Unique identifier (UUID)
user_id	Reference to the user who logged the item
date	Date the item was logged (YYYY-MM-DD)
item_name	Confirmed item name (string)
category	Normalized category (string, e.g. “coffee”, “electronics”)
price	Confirmed price in USD (number)
rarity_tier	Derived from price: Common / Uncommon / Rare / Epic / Legendary
cost_score	0–50, based on rarity tier
uniqueness_score	0–50, based on category frequency at time of logging
happiness_value	cost_score + uniqueness_score (0–100)
original_image_url	Storage path/URL for the original photograph
styled_image_url	Storage path/URL for the Nano Banana output
created_at	Timestamp of record creation

5.2 Storage Requirements

Each item record stores two images: the original photo and the Nano Banana stylized output. At an estimated average of 1–2 MB per image pair (compressed), a user logging 3–5 items per day would accumulate roughly 3–10 MB per day, or 90–300 MB per month. Cloud storage (e.g., S3, Google Cloud Storage) with CDN delivery is recommended.

6. Architecture & Technology

6.1 Three-Layer Architecture

- UI Layer: React (web) initially, with planned migration to React Native for iOS and Android. State management via Zustand or a shared hooks layer to ensure portability.
- Service Layer: Handles the pipeline orchestration (identify → confirm → generate → score), API calls to AI providers, and application state. This layer should be framework-agnostic to support the React Native migration.
- Backend / AI Layer: API routes that proxy requests to Nano Banana (via fal.ai or Gemini API), vision LLMs, and handle image storage. Keeps API keys server-side.

6.2 React Native Migration Considerations

The web app is the initial target, but the product's core interaction (camera capture) is inherently mobile. To ease the future migration, the service layer and API integration logic should be kept in plain TypeScript with no React-specific dependencies. UI components will need to be rewritten for React Native, but the business logic, scoring calculations, and API clients should be directly reusable.

7. Future Considerations

- Streaks & multipliers: Reward consecutive days of logging with score multipliers to boost retention.
- Weekly/monthly aggregates: Summary views showing total happiness, top items, rarity distribution over longer periods.
- Social features: Share loot drops with friends, compare daily scores, or trade items.
- Intentionality bonus: Optional note field where users describe why a purchase made them happy, earning bonus points for reflection.
- Category rarity evolution: As the user's history grows, dynamically adjust which categories feel "rare" versus "common" based on their personal purchasing patterns.
- Cost model optimization: Use cheaper model tiers (Nano Banana Fast) for Common items and reserve Pro/Ultra for Rare+ items to reduce per-user cost.